

Applying AspectJ to J2EE Application Development

Nicholas Lesiecki, *Video Monitoring Services of America*

Aspect-oriented programming and the AspectJ language can reduce code size and defects while increasing understandability.

Aspect-oriented programming (AOP) is a technique that allows programmers to modularize crosscutting concerns (behavior that cuts across the typical divisions of responsibility, such as concurrency or security). For example, a typical application might have a security policy that prevents a user from executing a number of operations unless the user has the correct privileges. Often, these sensitive operations would live on different objects. Because the security policy doesn't fit neatly

into one object, but touches several objects, it's a crosscutting concern. To implement this concern without aspects, a development team would need to modify each sensitive operation so that it first checks the user's access rights. This solution is difficult to modify, understand, and change. With AOP, a programmer could write an aspect to define a set of sensitive operations and that says in essence: "before each such sensitive operation, check the user's access level." This captures the entire security policy in a reusable module. (For more information on AOP, see my introductory article, "Improve Modularity with Aspect-Oriented Programming,"¹ *The AspectJ Programming Guide*,² or the sidebar.)

In January 2004, Video Monitoring Services of America (VMS) began to investigate the adoption of AOP into the development of Adbase, their Java 2 Enterprise Edition-based application. Adbase supplies a Web-based search and e-commerce interface on VMS's library of advertising data. In many ways, it's typical of J2EE application devel-

opment. It uses a wide range of J2EE technologies, both commercial and open source. The Adbase team varied from four to seven full-time developers during the time frame covered by this article. We followed a modified version of Extreme Programming with a focus on emergent design and adherence to XP practices such as programmer testing and pair programming.

The Adbase team was interested in AOP because we were familiar with its benefits. As the project's principal engineer and coauthor of *Mastering AspectJ*,³ I championed the investigation and subsequent adoption of the technology. Crosscutting concerns exist in a wide variety of problem domains and, based on my experience with aspects, I could identify a number of areas in Adbase where I thought aspects would improve the code's modularity, flexibility, and clarity. We were encouraged in our investigation by reports of aspects being used successfully to improve separation of concern and component reuse in large commercial codebases.⁴

We selected AspectJ as our aspect implementation because it's a general-purpose aspect-oriented extension to the Java language. As a language extension, it allowed for incremental experimentation with aspects. Most of the project could remain implemented with object-oriented code, with select crosscutting concerns implemented in aspects. Several competing aspect implementations for Java existed at the time of our adoption, but we deemed them too immature for our consideration.

Applications

We began our adoption process with two significant experiments: an error-logging aspect and an application-specific aspect. The experiments exposed the team to the technology in a low-risk way, and thus increased familiarity and confidence. Having gained significant experience with AspectJ, the team next felt ready to approach a more ambitious problem: relationship-management aspects.

Error logging

Our investigation of AspectJ began with a typical first use. Our application had been plagued by poor exception handling in a third-party library. Our problem was pseudocode of the form

```
public void evaluate(String arg){
    try{
        useReflectionToCallInto-
            Application(arg);
    }catch(Exception e){
        throw new JspException(
            e.getMessage() );
    }
}
```

This type of handler block discards the stack trace from the original exception, leaving application developers mystified about the underlying cause of the problem.

The Adbase team decided to use AspectJ's bytecode-weaving capabilities to advise handler blocks of this type. Because the offending library was open source, we could have patched the library's source code to correct the offending handlers. However, we would have then had to either submit the patch and ensure that our code was general enough to serve all library clients or maintain our modifications for new versions of the library.

Glossary of AOP Terminology

Aspect-oriented programming, like any new programming technique, introduces its own set of concepts and terms to refer to them. Here are explanations for some of the terms used in this article. My definitions are informed by my focus on AspectJ; other AOP implementations may have their own terms or construe some meanings differently.

Advice—declaration of code to execute at each join point picked out by a pointcut. You can think of a pointcut as the definition of an interesting set of events in a program and advice as the reaction to those events. Advice has access to context exposed by the pointcut. Advice comes in three flavors depending on how it executes with respect to the join point. *Before* advice runs before the join point it operates over. *After* advice runs after a join point completes (the most commonly used type of after advice is *after returning* advice, which only executes if the join point completes normally). *Around* advice replaces the join point it operates over. In around advice, the computation of the original join point can be executed with the special `proceed( ... )` syntax.

Aspect—a special type (much like a class in Java) that encapsulates pointcuts, advice, intertype declarations, and other crosscutting features.

Control flow—some pointcuts and aspects are defined in terms of the control flow of a join point. For example, the control flow of a method execution join point consists of all the other join points that occur between the start and end of that method's execution. If the method `foo()` calls `bar()` and `baz()`, the executions of `bar()` and `baz()` are said to be in the control flow of `foo()`.

Intertype declarations—declarations in an aspect that affect a program's static type structure. An example is extending a class with a new member variable or method.

Join point—a well-defined point in a program's execution. A method execution is a join point, as are field access and modification, among others.

Perflow() aspect—an aspect whose life cycle is bound to a given control flow. The AspectJ runtime creates the aspect at the beginning of the control flow and releases it at the end of the control flow. Among other things, this allows the aspect to maintain state for the duration of the control flow.

Pointcut—a selector for join points. For example, you could write a pointcut that matches all of the method executions on the class `Foo` with the following pointcut: `execution(* Foo.*(..))`. As you can see, pointcuts can take advantage of wildcards in their definition. They can also be composed with operators such as `||`, `&&`, and `!`. For example, `execution(* Foo.*(..)) || execution(* Bar.*(..))` selects all method executions on `Bar` or `Baz`. AspectJ lets you define your own pointcuts composed of combinations of primitive pointcuts (such as `execution()`) supplied by the language. Pointcuts can also expose context (such as argument values to a method call) that can be used in advice.

To improve the error behavior, we crafted a pointcut (see the "Glossary" sidebar) to identify handler blocks in an area of the code that we knew from experience was troublesome.

```

pointcut basketChange(ShoppingBasket basket):
(
    execution(public void addItem(Item, ..))
    || execution(public void remove*(..))
)
&& this(basket);
(a)

private boolean ShoppingBasket.isDirty = false;
after(ShoppingBasket basket) returning :
basketChange(basket)
{
    basket.isDirty = true;
}
(b)

pointcut chargeableReads() :
(
    execution(public * Chargeable+.getCharges(..))
    || execution(public * Chargeable+.getTotal(..))
    || execution(public * Chargeable+.hasCharges(..))
)
&& !cflow(execution(public * Pricer.*(..)));
(c)

before(ShoppingBasket basket) :
chargeableReads() && this(basket)
{
    if(basket.isDirty){
        pricer.price(basket);
        basket.isDirty = false;
    }
}
(d)

```

Figure 1. Basket-repricing aspect operations: (a) pointcut for selecting a dirtying operation, (b) intertype field and advice for marking a basket dirty, (c) pointcut that selects reads of the basket's pricing state, and (d) advice for repricing the basket.

Then we added before advice to log the exception argument to the handler block.

Our first aspect's net effect was positive. Once we saw stack trace information in the application log, it took us only moments to solve errors that formerly required a trip to the debugger or careful source code analysis. The exposure to AspectJ in this limited context encouraged further exploration.

Application-specific aspects

Next, the Adbase team experimented with other auxiliary aspects (such as aspects for gathering quick-and-dirty performance statistics). Finally, educating itself about AOP and

AspectJ, the team decided to add AspectJ to the development process with a pilot project. We deliberately limited the proposed aspect's scope so we could reincorporate a traditional implementation of the feature should the experiment prove unsuccessful. The aspect managed administrator workflow for new meta-data added to the system.

Although we encountered some challenges, the pilot was successful and the aspect became the first of many limited-scope, application-specific aspects that we wrote in the subsequent months.

A good example of an application-specific aspect is the one we wrote to handle the dynamic repricing of an online shopping basket in response to content or delivery-option changes. Basket pricing in Adbase is complex. Different charges apply to items, groups of items, and the order as a whole. Some charges depend on the presence of other charges. A change to a setting in one part of the basket can require the recalculation of the price of items in another part. Because so many operations and objects can impact the basket's chargeable state, repricing was a clear crosscutting concern.

Our basket-repricing aspect tracks state changes with a simple Boolean flag. Any operation that can affect the basket's price sets the flag to true. Before client code reads pricing information from the basket, the aspect reprices the basket and sets the flag to false.

The aspect selects dirtying operations with pointcuts such as that in figure 1a. It then uses an intertype field and advice to mark the basket dirty after each state change, as figure 1b shows. Next, the aspect selects operations representing reads of the basket's pricing state as shown in figure 1c. Finally, using before advice, the aspect reacts to each read operation by repricing the basket as illustrated in figure 1d.

Each cart component that can receive charges implements the Chargeable interface. This interface facilitates picking out reads across three different classes. The `chargeableReads()` pointcut uses the `cflow()` primitive pointcut to exclude reads occurring as a result of any method on the Pricer object. This prevents a recursive call to the advice during repricing.

The basket-repricing aspect is one of Adbase's most successful application-specific aspects. Because of this aspect (and another closely related aspect), `ShoppingBasket` and its associated items contain almost no pricing-

related code. This lets them better represent their primary role (a structured collection of items for purchase). It also lets developers reason about and evolve the pricing and repricing behavior in isolation, without having to inspect or modify the various price-triggering events.

By encapsulating the pricing logic, the aspect solution added extensibility and composability to the basket package. (We could employ alternative pricing strategies with fewer changes to the code.) It also increased the code base's agility. Because we could easily implement reprice strategies touching a dozen or more operations in a single location, we felt free to experiment with alternative designs.

Relationship management

The Adbase team developed its most advanced set of aspects in response to a new persistence solution. We were abandoning Enterprise JavaBeans' (EJB) Container Managed Persistence in favor of the lighter-weight Hibernate. Unfortunately, during the changeover, we discovered that Hibernate lacked the container-managed relationships that we had begun to take for granted with CMP. We termed the missing feature *bidirectional relationship propagation*. Figure 2 shows a model of two persistent objects in a typical relationship.

If we implemented the model in figure 2 in pure Java code, calling `child.setParent(someParent)` on the child wouldn't affect `someParent`'s children collection. However, if an EJB container had managed that relationship, calling `child.setParent(someParent)` would result in the equivalent of a call to `someParent.getChildren().add(child)`. The container would propagate the link to the other side of the relationship, making the relationship navigable in both directions. Although Hibernate does this as it reads objects from or writes them to the database, it doesn't propagate the relationship during the object's general use by an application.

Using aspects to address a weakness in or to extend an existing application or framework raises questions. For example: Can we use the framework another way and get the same result? Would it be a better to modify the framework directly?

According to Hibernate's own documentation, the preferred solution to our problem is to write code in each relationship setter to traverse and propagate the relationship. This is exactly

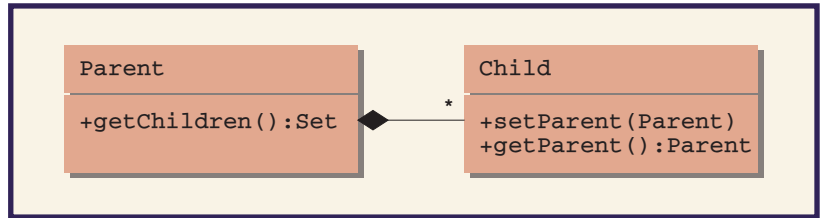


Figure 2. Model of persistent objects in a relationship.

```

public aspect RelationshipManagement {
    //...
    pointcut setToOne(Persistent self, Persistent other) :
        call(void set*(Persistent+)) && target(self) &&
        args(other);

    before(Persistent self, Persistent other) : set
        ToOne(self, other) {

        String methodName = //...obtain using AspectJ
            reflection

        BidirectionalRelationHelper.reciprocateToOne
            (methodName, self, other);
    }
}
  
```

Figure 3. A pointcut and advice that implement part of the bidirectional relationship behavior.

what we hoped to avoid. Also, extending or patching Hibernate presented problems. It was our persistent objects whose behavior needed modification (Hibernate already "did the right thing" when loading the objects). We decided that if we could test and deploy the aspect solution in less than two weeks, we'd stick with it.

We began our implementation with one-half of the desired behavior. To automatically add a child object to a parent's collection after a call to `setParent()`, we first had to identify the `setParent` join points. To do so, we wrote the pointcut in figure 3, which explicitly assumes that a method call signifies the start of a relationship if it meets three criteria:

- It begins with the letters s-e-t.
- The method call's target implements an interface we created called `Persistent`.
- The sole argument is also a `Persistent`.

The pointcut exposes both the target and the argument object for use within the advice. Because the pointcut relies on an identifier pattern, we accepted the risk that the pointcut might match join points incorrectly (such as

Figure 4. Propagating changes to a Parent's children collection:
(a) around advice wraps the returned collection in a special marker collection
(b) after-returning advice propagates changes made to the marked collection.

```

pointcut persistentSetAccess(Persistent self) :
    call(java.util.Set Persistent+.get*())
    && target(self) && managedRelationshipAccess();

java.util.Set around(Persistent self) :
    persistentSetAccess(self) {
        String methodName = //...obtain using AspectJ
            reflection
        return new RelationshipAwareSet(methodName, self, proceed(self));
    }
(a)

//pointcut relationshipAdd() not shown
after(RelationshipAwareSet set, Object o) returning(boolean added) :
    relationshipAdd(set, o) {
        //'source' is a reference to the originating object (stored by the set)
        if(added){
            BidirectionalRelationshipHelper.reciprocateToMany(set.
                propertyName, set.source, (Persistent)o, true);
        }
    }
(b)

```

`Account.settle(Account))`. We judged this risk to be acceptably low because of the other join point criteria's limiting effect. When making judgments like this, we relied both on our own experience and on design techniques emerging from the AOP community. For example, Adrian Colyer's blog entry, "I Don't Want to Know That ... (Writing Robust Pointcut Expressions)," covers just this sort of design decision (see www.aspectprogrammer.org/blogs/adrian/2004/08/i_dont_want_to.html).

The advice in figure 3 extracts the setter method's name using reflective information provided by AspectJ. The advice then delegates to a helper class to navigate the metadata provided by Hibernate. The helper class retrieves the children collection and adds the child object (referenced in figure 3 by `self`) to that collection. We localized the logic in a helper class to keep the definition of the crosscutting structure in one module (the aspect) while letting the details of Hibernate metadata traversal reside in another. This allowed for easier testing of the corner cases of metadata traversal because we could easily feed the appropriate arguments to the helper class. This decision also reduced compilation time. Because changing an aspect requires a full build, splitting the im-

plementation let us change the traversal logic without losing incremental compilation.

At this point we had only solved half of our problem. We still needed to propagate changes to a parent's children collection to the child side of the relationship. Here we faced a problem. The parent object could pass its children collection to another part of the application, which might then modify the collection. We needed a way of marking the collection so that we could identify future modifications to it. We did this by using an aspect to wrap these collections in a special marker collection that we wrote.

To accomplish this, we again select relationship getters with a pointcut based on a combination of name and type information. The aspect then uses around advice to modify the affected join points' return value (we obtain the return value through a call to `proceed()`) as shown in figure 4a.

After we had marked children collections, we could affect them using advice. The after-returning advice shown in figure 4b inspects the return value of `add()` on a `RelationshipAwareSet`. If the addition was successful, the advice uses the helper class to propagate the relationship.

After implementing these two aspects, we

```

public aspect RelationshipSevering percall(call(* Session+.delete(..)) {
    private Set alreadyDeleted = new HashSet();

    pointcut onDelete(Persistent objectBeingDeleted) :
        execution(public boolean Lifecycle+.onDelete(Session+) &&
            this(objectBeingDeleted);

    after(Persistent objectBeingDeleted) : onDelete(objectBeingDeleted) {
        BidirectionalRelationHelper.
            severRelationships(alreadyDeleted, objectBeingDeleted);
        alreadyDeleted.add(objectBeingDeleted);
    }
}

```

Figure 5. Code for severing relationships on object deletion.

encountered a set of test failures. If the application deleted a child object, Hibernate expected the application to remove the object from the parent's collection. So we added another dimension to our solution. To detect deletion events, we wrote a pointcut that used a Hibernate marker interface. By writing the pointcut in terms of the `Lifecycle.onDelete()` method, we leveraged Hibernate's definition of a deletion event. After each such event, our aspect used a helper class to iterate over all of the doomed object's relationships and removed the dying object from them.

This solution hit a snag, however. Hibernate, like other persistence frameworks, manages cascade deletions. If our severing aspect affected a relationship in the middle of a cascade deletion, Hibernate would detect the condition and throw an exception. To exclude cascade deletions, we used a percall aspect. Percall aspects only exist during the control flow of specified join points. This feature allowed the aspect to maintain a set of all the objects that had already been deleted during a given cascade. The aspect passed this set of already dead objects to the helper class, which could then decline to sever a relationship to a deleted object. Figure 5 shows the code for this aspect. Notice that the percall pointcut in the aspect declaration specifies that the aspect should be instantiated once for each call to one of Hibernate's `Session.delete()` methods. Most object deletions in Hibernate originate from such a call.

In our project, Hibernate persists 63 classes containing 42 methods of the form `child.setParent()` and 39 of the form `parent.`

`getChildren()`. Another 11 relationships are modified through another mechanism, which our solution also addressed (but is not covered here). These metrics give a sense of the aspects' impact. Implementing the solution by manually inserting a call to the relationship helper would have involved modifying 92 disparate sites. It's not hard to imagine our team missing five or 10 sites and stumbling across a bug later on. Furthermore, without the tangled calls to the bidirectional relationship helper, the persistent objects are easier to read, write, and understand. A developer can program Hibernate-enabled objects that participate in bidirectional relationships with no special effort. So, we could say that the aspects improve the system's maintainability and understandability.

Team reactions to AOP

As a whole, the team's reaction to AOP and AspectJ was positive. Even programmers inexperienced with object-oriented programming grasped the basic concepts easily, and programmers moved quickly from being passive observers to participants in or initiators of pairing sessions involving AspectJ. About six months elapsed between the VMS's initial investigation of AOP and the development of the Hibernate aspects described in this report. This was calendar time, not effort time, as the team spent (and still spends) only about 5 to 10 percent of its development effort on AOP.

As the team's familiarity with the technology has grown, team members have become more excited about the potential for further applications. It's not uncommon to hear, "we could use an aspect to do that," floating through the de-

The combination of testing and pairing gave us the confidence to use our application as a learning environment.

velopment area. Some members have become champions of the technology. One programmer is investigating adding AspectJ to another Java project, and our manager acquired a custom license plate reading “AspectJ.”

For programmers less invested in AOP, its noninvasive nature has been a blessing. Developers can craft components that participate in aspect-oriented behavior without making numerous design concessions. It’s possible to work on the Adbase project without significant AspectJ experience; indeed, some programmers have yet to write a single pointcut. Compared to invasive frameworks such as EJB or Java Server Pages, AspectJ has weighed lightly on the project’s design and development.

Despite the generally positive reaction, AOP has had some challenges at VMS. Because AOP demands a new way of thinking about software construction and a new language in which to express that thinking, it meets a fair amount of resistance. My team has been no exception. During initial presentations, the audience clearly saw the technology’s potential, but was apprehensive about the learning required to leverage it. As time passed, some of the apprehension dissipated. However, new team members approaching the technology for the first time experienced similar apprehension.

Although our familiarity with AspectJ and AOP has grown, we’re still in the process of altering our mental models to accommodate aspect orientation. Scott Segal, the Adbase team manager, put it like this:

The most difficult part of adopting AOP was making the mental shift from thinking of AOP as a technique to solve a narrow set of problems to embracing AOP as part of our standard approach to solving all problems. When looking at a problem now we try to see how the problem naturally separates into concerns and implement the crosscutting concerns with aspects.

Although programmers can remain mostly oblivious to aspects (especially orthogonal ones), to work effectively on an aspect-oriented project, they must know when an aspect could be advising code they’re working on. Without this awareness (and general knowledge of how AspectJ works), aspectual effects might confuse or surprise programmers. The less orthogonal the aspect, the more programmers must be aware of it.

I call one flavor of this problem “aspects as

a red herring.” This problem arises when programmers uninformed with an aspect’s development encounter a problem in that aspect’s domain. Suspicious that the aspect might be involved, but not knowing its details, they feel overwhelmed by having to understand the aspect before solving the problem. The aspect might not actually be involved, but its possible presence can be a confounding factor.

Adoption process

By limiting the scope of our initial aspects, we were able to learn concepts, best practices, places where aspects made sense, and places where they didn’t. For example, we’ve replaced several aspects with nonaspect solutions after reviewing the resulting design. Our early experiments gave us the confidence and experience to approach the relationship management aspects—the most challenging and general set of aspects developed yet. The incremental approach both diffused adoption cost over several months and let concepts and ideas soak into the team consciousness.

Unit and integration tests

The XP practice of *automated testing* eased our adoption of AspectJ. At the time we began adoption, we had amassed a suite of several hundred unit and integration tests. Automated testing helped us by

- letting us verify an aspect’s implementation during its development,
- letting us quickly detect side effects or notice when a refactoring had broken an aspect, and
- forcing us to learn about robust AOP.

Pair programming

Pair programming proved invaluable for transferring knowledge about AOP and AspectJ. A typical early pair session placed me with a less-experienced developer as we tried to write an aspect to address a specific concern. When we came to new language features or a new aspect-oriented concept, I explained the concept and we continued with the implementation.

As team experience grew, aspect-oriented pairing sessions began between other team members. We used our shared workspace to exploit the “expert in earshot” effect (see Alistair Cockburn’s Wiki entry at <http://c2.com/cgi/wiki?ExpertInEarshot>). Team members strug-

gling with a pointcut's nuances could easily get my attention for a quick resolution.

The combination of testing and pairing gave us the confidence to use our application as a learning environment. We were able to try out different ideas and easily realize them within a familiar context. This helped us learn AOP more rapidly and confidently than we otherwise might have.

Refactoring

In our XP environment, we refactor often, and aspects are no exception. Just as object-oriented code can vary in its readability and expressiveness, aspect-oriented code can be clear or confusing. Accordingly, we often revisited our aspects (for example, when we modify them to add a new feature) and attempted to express their intent more clearly. Many of the habits, practices, and guidelines for refactoring object-oriented code translate well to aspect-oriented code. In addition to refactoring aspects, we also refactor our code to add aspects where warranted, and to remove aspects when the program's intent is better expressed without them.

Tool support

Over the last five years, automated tools have become central to development in the mainstream programming community. An increasing number of programmers (the VMS team included) use an integrated development environment for day-to-day coding. As time passes, it becomes increasingly difficult to disentangle a programming language from its supporting toolset. This contention applies especially to AspectJ. We primarily used the fast-evolving AspectJ Developer's Toolkit (AJDT) Eclipse plug-in available at www.eclipse.org/ajdt.

Crosscutting views

When working on a module, a programmer must be somewhat aware of crosscutting aspects (or at least be able to transition quickly from oblivious to aware), especially when the aspect is tied closely to the module's core functionality (for example, repricing a basket). Similarly, in the "aspects as a red herring" situation, a programmer must be able to confidently rule out (or find) potentially troublesome aspects.

AJDT provides two tools that help make a program's crosscutting structure understandable:

- The *cross-references view* lets developers view all of the join points affected by a given piece of advice (it provides similar facilities for intertype declarations). This lets them easily analyze an aspect's potential effect. When writing a new pointcut, for instance, the cross-references view alerts you if an error prevents a pointcut from matching any join points. Similarly, a quick scan tells you if the pointcut matches unintended points.
- When a programmer views a class (or an aspect) in Eclipse's editor, *gutter annotations* indicate if advice applies to the current program element. By providing a clear indication of aspect impact and an easy means to navigate back to the aspect, AJDT's gutter annotations have helped us diagnose aspect-related problems. They also serve as unobtrusive reminders that aspects contribute to the code's total behavior.

Debugging support

Although the debugging support in AspectJ exhibits a few problems—such as the occasional inability to set breakpoints without a rebuild and some confusing stack frames to step through—the current implementation operates largely as desired. In other words, you can set breakpoints in classes and step into advice that applies to them. You can also set breakpoints in advice. As we developed the relationship-management aspects, we frequently used the debugger to help us understand the code's dynamic behavior.

Compilation speed

The biggest challenge to VMS's further adoption of AspectJ is the significant increase in incremental compilation times when using it. AspectJ places two important restrictions on incremental compiles:

- Incremental compilations using AJDT (in the Adbase project) invariably take a couple of seconds longer than the near-instant times of pure Java builds.
- Changes to an aspect require a full rebuild (performed automatically by AJDT even in incremental mode).

These two types of slowdown noticeably affect development velocity. Our XP-based development process relies on unit tests and in-

The biggest challenge to VMS's further adoption of AspectJ is the significant increase in incremental compilation times.

About the Author



Nicholas Lesiecki is a programming instructor and software engineer at Google. His research interests include agile methods, aspect-oriented programming, and test-driven development. In addition to coauthoring *Mastering AspectJ* (Wiley, 2003), he is a member of AspectMentor. He has a BA in English literature from Amherst College. He is a member of the ACM. Contact him at 100 N. Whisman Rd. #215, Mountain View, CA 94043; ndlesiecki@apache.org.

cremental development. Ideally, we'd modify code, compile it, and execute the relevant unit tests several times per minute. Adding even a few seconds of delay to the cycle slows development on two fronts. First, the compilation takes time. Second, human attention can wander during the wait interval, and it can take time to recontextualize after the compilation. This problem is particularly pronounced for the full builds, which tempt the programmer to switch to another task entirely (such as email or Slashdot headlines). One of VMS's most talented programmers, despite having positive experiences on the Adbase project and having designed an aspect-oriented testing framework, resists using AspectJ in his other projects because of these delays. According to him, "The benefits realized by AOP over pure object-oriented approaches do not outweigh the drawbacks of productivity loss and impediments to a test-driven development approach, at least in a fairly small application."

The good news is that improving performance is a priority for the AspectJ developers.

Refactoring and Java editing

Although AJDT brings most of the functionality of a good Java editor to AspectJ (code completion and syntax highlighting, for example), important areas are missing. These feature holes have tripped up developers used to Eclipse's rich functionality. Code completion works only in certain contexts, for example, and the "open type" shortcut doesn't apply to aspects.

More importantly, refactorings initiated from Java don't affect aspects, making wide-ranging refactoring a more delicate and manual process.

VMS considers its adoption of AOP successful. The key barriers to further adoption within the company are in the area of tool support rather than training or

conceptual understanding. In particular, compilation times incur a productivity penalty that offsets aspect-oriented development's general benefits. The AJDT environment's capabilities also lag behind those of Eclipse's general Java capabilities. Luckily, AspectJ's developers are giving these areas significant attention.

Despite these barriers, aspects have noticeably improved the Adbase application's "ilities." The application exhibits improved resilience to change and ease of understanding in areas where we've successfully applied aspects. AOP has also reduced the amount of needless duplication and busy work necessary when adding functionality to key areas—for example, adding a new persistent relationship.

The Adbase team learned that its agile development approach handles aspect-oriented development's challenges well. Our methodology limited risk, encouraged beneficial skepticism, and maximized knowledge transfer. ☞

Acknowledgments

My thanks to Scott Segal, Angus McIntyre, Chad Woolley, and the other members of the Adbase team for their reviews and advice.

References

1. N. Lesiecki, "Improve Modularity with Aspect-Oriented Programming," *IBM DeveloperWorks* 2002, www-128.ibm.com/developerworks/java/library/j-aspectj.
2. The AspectJ Team, *The AspectJ Programming Guide*, Xerox Palo Alto Research Center, 2004, <http://eclipse.org/aspectj/doc/released/progguide/index.html>.
3. J. Gradecki and N. Lesiecki, *Mastering AspectJ: Aspect-Oriented Programming in Java*, John Wiley & Sons, 2003.
4. A. Colyer et al., "Using AspectJ for Component Integration in Middleware," *Proc. Object-Oriented Programming, Systems, Language, and Applications (OOPSLA 03)*, ACM Press, 2003, pp 339–344.

For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/publications/dlib.